

Formalizing Type Safety for Call-by-Value Evaluation in the Simply Typed Lambda Calculus with Booleans in Lean 4

Soma Saito

Abstract

Type safety of a typed lambda calculus ensures that closed, well-typed terms do not get stuck during evaluation; it is conventionally established by preservation and progress. This paper completes the call-by-value type-safety metatheory for the `STLCextBool` development in the Lean 4 `Metatheory` repository. The development already provides subject reduction and progress for an unrestricted one-step relation `Step`, and determinism and confluence for the call-by-value relation `CBVStep`. We prove that each `CBVStep` is a `Step` (`CBVStep.to_step`), reuse the existing subject-reduction theorem to derive `CBVStep.preservation`, and prove `CBVStep.progress` separately so that it respects the call-by-value evaluation order. Finally, using the generic reflexive-transitive closure `Rewriting.Star`, we formalize multi-step preservation and a no-stuckness form of type safety. Thus, every term reached by zero or more call-by-value steps from a closed, well-typed term is either a value or can take a further call-by-value step.

1 Introduction

The metatheory of typed lambda calculi is a foundational topic in programming language theory [2]. Type safety is especially important: it states that a well-typed term cannot get stuck while it is evaluated. It is usually decomposed into two results. *Preservation* states that a one-step reduction preserves a term's type. *Progress* states that a closed, well-typed term is either already a value or can take a next step.

Mechanically checking these results in a proof assistant is an effective way to make proofs about type systems and operational semantics precise [3, 6]. Case analyses and dependencies on auxiliary lemmas that are often implicit on paper must be made explicit in a formalization. Consequently, the formalization records not only that a theorem holds, but also exactly which definitions and previously established facts its proof uses.

This explicitness matters even for familiar metatheorems. A paper proof may silently appeal to substitution, inversion, or a canonical-forms argument. In Lean, the statement on which each appeal relies and the shape of every relevant case must be available to the kernel. The resulting development is therefore also useful as a map of the dependency structure behind type safety.

This paper studies `STLCextBool` in the `Arthur742Ramos/Metatheory` repository [4, 5]. It is a Lean 4 formalization of the simply typed lambda calculus extended with Boolean constants and conditionals. It defines types, terms, typing, reduction relations, and their metatheory. In particular, it contains both an unrestricted one-step relation `Step` and a call-by-value one-step relation `CBVStep`.

The original development contains `subject_reduction` and progress for unrestricted `Step`, as well as results that values cannot take a call-by-value step, determinism, and confluence for `CBVStep`. What was still useful to state explicitly was preservation, progress, and multi-step type safety for `CBVStep`. Unrestricted progress merely shows that a term admits

some unrestricted reduction; it does not show that the reduction follows the call-by-value evaluation order.

The existing unrestricted theory already establishes that type checking is stable under its permissive reduction relation. The call-by-value theory had important operational facts—values do not step, and the relation is deterministic and confluent—but did not yet expose the standard type-safety theorems under its own name. This distinction is material: a theorem about an arbitrary next step does not automatically become a theorem about the next step selected by an evaluation strategy.

Our goal is to complete this call-by-value type-safety layer. We first show that every **CBVStep** is also a **Step**. This lets us reuse the existing unrestricted subject-reduction theorem to derive call-by-value preservation. Progress is instead proved directly from the call-by-value evaluation order. We then use the generic reflexive-transitive closure **Rewriting.Star** to establish multi-step preservation and type safety: any term reached by zero or more **CBVSteps** from a closed, well-typed term is a value or can take a further **CBVStep**.

The contribution is intentionally narrow. It does not redesign the target language or replace its operational semantics. Instead, it makes the already defined call-by-value relation usable as a complete type-safe evaluation relation, while keeping the proof close to the existing repository structure.

Section 2 reviews the background. Section 3 describes the target formalization and its existing results. Section 4 presents the added **CBVStep** theorems, Section 5 discusses proof structure and scope, and Section 6 concludes.

2 Background

This section reviews the typed calculus, its small-step operational semantics, the call-by-value evaluation order, and the standard components of type safety.

2.1 Simply Typed Lambda Calculus

The simply typed lambda calculus (STLC) is a basic typed computational calculus. It has types, terms, contexts, and typing judgments. A judgment

$$\Gamma \vdash M : A$$

states that term M has type A in context Γ . A context assigns types to variables; for example, $x : A, y : B \vdash M : C$ says that M has type C when x and y are available at the indicated types.

Typing does more than classify final answers. It constrains the forms a term may take during computation. In particular, it rules out applying a Boolean as a function and selecting a conditional branch with a non-Boolean guard. Those constraints are what make the progress argument possible.

Function types $A \rightarrow B$, lambda abstraction, and application are central constructs. A function of type $A \rightarrow B$ consumes an A and produces a B . The usual typing rules are

$$\frac{\Gamma, x : A \vdash M : B}{\Gamma \vdash \lambda x. M : A \rightarrow B} \quad \frac{\Gamma \vdash M : A \rightarrow B \quad \Gamma \vdash N : A}{\Gamma \vdash M N : B}.$$

We consider **STLCextBool**, which adds the type **Bool**, the terms **true** and **false**, and conditionals

$$\text{if } M \text{ then } N_1 \text{ else } N_2.$$

The conditional has type A when its guard has type Bool and both branches have type A :

$$\frac{\Gamma \vdash M : \text{Bool} \quad \Gamma \vdash N_1 : A \quad \Gamma \vdash N_2 : A}{\Gamma \vdash \text{if } M \text{ then } N_1 \text{ else } N_2 : A}.$$

The equal branch type is essential: no matter whether the guard evaluates to true or false, selecting a branch retains the type promised by the original conditional. This is the Boolean-specific part of the preservation argument; the rest follows the usual STLC account.

2.2 Small-Step Semantics and Call by Value

Operational semantics describes how terms compute. In small-step semantics, one-step reduction $M \longrightarrow N$ says that M takes one computation step to N . Its reflexive-transitive closure describes zero or more steps: a term reaches itself in zero steps, and a multi-step path can be extended by one more step. The formalization represents this closure by **Rewriting.Star**.

Writing reduction as a relation, rather than immediately as a recursive evaluator, lets the formalization state general properties of paths of computation. In particular, the reflexive case represents zero evaluation steps, while the step-extension case represents one additional operational transition. Both cases are needed when type safety quantifies over all reachable terms.

STLCextBool defines unrestricted **Step** and call-by-value **CBVStep**. The latter fixes an evaluation order. For an application $M N$, it first evaluates M , then evaluates N after M is a value, and performs beta reduction only when the function is a lambda abstraction and the argument is a value. A conditional first evaluates its guard; it selects the then branch only for **true** and the else branch only for **false**. It never evaluates a branch before its guard.

Thus an unrestricted step need not be a valid call-by-value step. In particular, unrestricted progress does not by itself establish call-by-value progress. The latter must show that the next step obeys the specified evaluation order.

For example, an unrestricted relation may reduce an argument before the function position of the enclosing application has become a value. A call-by-value evaluator is not allowed to choose that redex at that point. The difference is not about whether the term can eventually compute; it is about which transition is justified as its immediate next transition.

2.3 Values and Type Safety

Values are terms that require no further computation. In **STLCextBool**, lambda abstractions, **true**, and **false** are represented by **IsValue**. Their role is essential in call-by-value evaluation: substitution into a function body occurs only after the argument has become a value.

Type safety says that a closed, well-typed term cannot reach a stuck state, that is, a non-value from which no next step is possible [7, 2]. Its two standard components are:

$$\Gamma \vdash M : A \wedge M \longrightarrow N \implies \Gamma \vdash N : A$$

for preservation, and

$$\emptyset \vdash M : A \implies M \text{ is a value} \vee \exists N, M \longrightarrow N$$

for progress. Together they ensure that each reachable non-value remains well typed and can continue to evaluate.

The multi-step formulation is often the most useful form in a mechanized development. One-step preservation alone types only the immediate successor; induction over the closure transports the typing derivation to every later state. Progress can then be reapplied at every such state to rule out stuck configurations throughout an evaluation, not merely at its start.

2.4 Mechanized Metatheory in Lean 4

Mechanized metatheory represents syntax, typing, reduction, and their properties in a proof assistant. Well-known teaching formalizations include Software Foundations in Coq/Rocq and PLFA in Agda [3, 6]. In Lean 4, syntax and relations can be given as inductive definitions and metatheorems can be checked by the kernel [1]. A completed proof without `sorry`, together with a successful module build, therefore validates the stated theorem.

We refer to formal objects by their Lean names, such as `CBVStep.progress` and `Rewriting.Star`. The results added in this work are

```
CBVStep.to_step,      CBVStep.preservation,
CBVStep.progress,    CBVStep.preservation_star,
CBVStep.type_safety.
```

The distinction between mathematical notation and Lean names is deliberate. The former makes the metatheory readable in the conventional presentation; the latter makes each claim directly traceable to a definition or theorem in the repository.

3 Existing Formalization

This section distinguishes the components already present in the Lean 4 `STLCextBool` formalization from the type-safety results completed by this work [4, 5]. Mathematical prose uses standard notation such as $\Gamma \vdash M : A$, while implementation objects are named by their Lean identifiers, including `Term`, `Ty`, and `HasType`.

3.1 Syntax and Typing

The existing development defines `Ty` in `Types.lean`, `Term` in `Terms.lean`, and `Context` and `HasType` in `Typing.lean`. It extends STLC with Boolean terms and conditionals. The usual conditional typing rule is

$$\frac{\Gamma \vdash M : \text{Bool} \quad \Gamma \vdash N_1 : A \quad \Gamma \vdash N_2 : A}{\Gamma \vdash \text{if } M \text{ then } N_1 \text{ else } N_2 : A}.$$

Thus evaluating the guard selects one of two branches that have the same type. The results below assume this existing syntax and typing relation; they do not modify either of them.

In the implementation, the use of a named typing relation also makes the boundary of the present contribution clear. The proofs in this paper start from existing `HasType` derivations and construct further derivations of the same relation. They introduce no alternative notion of value, typing, or contextual equivalence.

3.2 Unrestricted Reduction

The existing relation `Step` is an unrestricted one-step reduction. It includes beta reduction, the two conditional reductions, and rules for reducing relevant subterms of applications and conditionals. The development already proves the following preservation property as `subject_reduction`:

$$\Gamma \vdash M : A \wedge M \rightarrow N \implies \Gamma \vdash N : A.$$

It also contains multi-step preservation, `subject_reduction_multi`, and progress for unrestricted `Step`.

The unrestricted rules cover beta reduction, the two Boolean eliminations, and reduction under the relevant term constructors. Their flexibility is useful for general rewriting arguments, but it deliberately leaves open the choice of redex. Consequently, the existing theory gives a strong source of reusable facts without itself being a specification of a deterministic evaluator.

These theorems are useful but do not settle the call-by-value case. For example, unrestricted beta reduction need not require a value argument, and unrestricted reduction may contract subterms in positions that call-by-value does not permit yet. Hence unrestricted progress only supplies an arbitrary **Step**; it cannot directly supply a **CBVStep**.

The distinction also clarifies a subtle point about values. A lambda abstraction is a value for **CBVStep**, but an unrestricted **Step** may reduce inside its body. Thus the statement that a value is a stopping point must be read relative to the call-by-value relation, not as a claim that the term has no possible unrestricted redex.

3.3 Values and Canonical Forms

The existing predicate **IsValue** classifies lambda abstractions, **true**, and **false** as values. The associated stop-point claim is specifically about **CBVStep**: unrestricted **Step** may still reduce inside a lambda body. Two existing canonical-forms lemmas analyze a closed, typed value according to its type.

Lemma 3.1 (Existing canonical forms for functions).

*If $\emptyset \vdash V : A \rightarrow B$ and V satisfies **IsValue**, then V is a lambda abstraction. This is formalized as **canonical_forms_arr**.*

Lemma 3.2 (Existing canonical forms for Booleans).

*If $\emptyset \vdash V : \text{Bool}$ and V satisfies **IsValue**, then $V = \text{true}$ or $V = \text{false}$. This is formalized as **canonical_forms_bool**.*

These lemmas are the key inversion facts for call-by-value progress. When an application's function part is a value of function type, Lemma 3.1 makes it a lambda abstraction. When a conditional's guard is a Boolean value, Lemma 3.2 makes it either **true** or **false**.

These inversion results are precisely what turn typing information into an operational next step. A progress proof cannot select beta reduction merely from the fact that the function part is a value; it must first learn that a closed value at a function type is syntactically a lambda. The Boolean case has the same shape: a Boolean value must be one of the two constants that the conditional reduction rules recognize.

3.4 Call-by-Value Reduction

CBVStep is the existing one-step relation that fixes the call-by-value order. Its application rules, represented by **CBVStep.appL**, **CBVStep.appR**, and **CBVStep.beta**, are conceptually

$$\frac{M \rightarrow_{\text{cbv}} M'}{M \ N \rightarrow_{\text{cbv}} M' \ N} \quad \frac{\text{IsValue}(M) \quad N \rightarrow_{\text{cbv}} N'}{M \ N \rightarrow_{\text{cbv}} M \ N'} \quad \frac{\text{IsValue}(V)}{(\lambda x.M) \ V \rightarrow_{\text{cbv}} M[x := V]}.$$

It evaluates the function first, then the argument after the function is a value, and permits beta reduction only for a value argument.

For conditionals, **CBVStep.iteTrue** and **CBVStep.iteFalse** select the two branches, while **CBVStep.iteC** reduces the guard:

$$\frac{\text{if true then } N_1 \text{ else } N_2 \rightarrow_{\text{cbv}} N_1, \quad \text{if false then } N_1 \text{ else } N_2 \rightarrow_{\text{cbv}} N_2, \quad M \rightarrow_{\text{cbv}} M'}{\text{if } M \text{ then } N_1 \text{ else } N_2 \rightarrow_{\text{cbv}} \text{if } M' \text{ then } N_1 \text{ else } N_2}.$$

Only the guard is evaluated before a branch is selected. Our contribution is not a new definition of `CBVStep`; it is the missing type-safety layer over this definition.

The application and conditional rules encode the whole evaluation-context discipline needed here. They make no appeal to a separate evaluation-context syntax: the order is stated directly in the constructors. This is why the progress proof can proceed by choosing constructors in the same order as the intended evaluator would choose them.

3.5 Previously Available Results

Several important facts about `CBVStep` were already formalized. The lemma `CBVStep.value_no_step` states that a value has no call-by-value successor. Determinism is provided by `CBVStep.deterministic` and `cbv_deterministic`:

$$M \rightarrow_{\text{cbv}} N_1 \wedge M \rightarrow_{\text{cbv}} N_2 \implies N_1 = N_2.$$

The existing theorem `cbv_confluent` further establishes confluence: two multi-step `CBVStep` reductions from the same term can be extended to a common term.

Before this work, preservation, evaluation-order-respecting progress, `Rewriting.Star`-based multi-step preservation, and type safety were not stated as corresponding `CBVStep` theorems. The next section completes precisely those results, reusing the existing formalization wherever appropriate.

The separation between these previously available operational results and the new type-safety results is useful for maintenance as well. Determinism and confluence characterize the behavior of the reduction relation once steps exist; preservation and progress connect that behavior to typing and show that the prescribed evaluator is defined on every reachable well-typed configuration.

4 Main Results

We now present the type-safety results added for the pre-existing call-by-value relation `CBVStep`. Write $M \rightarrow_{\text{cbv}} N$ for one call-by-value step and $M \rightarrow_{\text{cbv}}^* N$ for `Rewriting.Star CBVStep M N`.

4.1 Embedding `CBVStep` into `Step`

The first result connects the two existing reduction relations.

```
theorem to_step {M N : Term} :
  CBVStep M N -> Step M N
```

Theorem 4.1 (Embedding of `CBVStep` into `Step`).

If $M \rightarrow_{\text{cbv}} N$, then $M \rightarrow N$.

The proof is a case analysis on a `CBVStep` derivation. Each constructor maps to the corresponding unrestricted constructor: `CBVStep.beta`, `CBVStep.appL`, `CBVStep.appR`, `CBVStep.iteTrue`, `CBVStep.iteFalse`, and `CBVStep.iteC` map to the corresponding `Step` rules. The value premise of call-by-value beta reduction is simply not needed by the more permissive `Step.beta`. The converse does not hold: an unrestricted step need not observe the call-by-value order.

This lemma is small, but it is the exact interface required for reuse. In the beta case, `CBVStep` carries the extra premise that the argument is a value, whereas the unrestricted beta rule accepts the same redex without that premise. In the congruence cases, the induction hypothesis converts the chosen subterm step and the matching unrestricted congruence

constructor lifts it to the enclosing term. The proof therefore records that the CBV rules are a restriction of the already established reduction relation.

4.2 One-Step Preservation

`CBVStep.preservation` derives preservation by reusing the existing unrestricted theorem.

```
theorem preservation {G : Context} {M N : Term} {A : Ty}
  (htype : HasType G M A) (hstep : CBVStep M N) :
  HasType G N A
```

Theorem 4.2 (One-step preservation for `CBVStep`).

If $\Gamma \vdash M : A$ and $M \rightarrow_{\text{cbv}} N$, then $\Gamma \vdash N : A$.

By Theorem 4.1, the step premise becomes an unrestricted `Step`. Applying the existing `subject_reduction` then gives the conclusion. The Lean proof has the essential form:

```
exact subject_reduction htype (to_step hstep)
```

This is a deliberate reuse of existing metatheory rather than a duplicate proof of preservation.

The result is uniform in the context Γ , the source and target terms, and the type A . It consequently applies not only to closed programs but also to every intermediate typing derivation used when constructing a multi-step proof. This generality is what makes it the appropriate one-step building block for the later no-stuckness theorem.

4.3 Call-by-Value Progress

Progress must be proved separately because its next step must respect the call-by-value order.

```
theorem progress {M : Term} {A : Ty}
  (htype : HasType ([ ] : Context) M A) :
  Or (IsValue M) (Exists fun N => CBVStep M N)
```

Theorem 4.3 (Progress for `CBVStep`).

If $\emptyset \vdash M : A$, then M is a value or there is an N such that $M \rightarrow_{\text{cbv}} N$.

The proof proceeds by cases on the typing derivation. The interesting cases are application and conditional. For $M N$, the induction hypothesis first considers the function part. If it steps, `CBVStep.appL` steps the whole application. If it is a value of function type, Lemma 3.1 identifies it as a lambda abstraction. The argument is then considered: if it steps, `CBVStep.appR` applies; if it is a value, `CBVStep.beta` applies.

More explicitly, the application case first obtains a disjunction for the function part. The stepping branch uses `CBVStep.appL`. In the value branch, the function has an arrow type, so canonical forms supplies a lambda body. The proof then obtains a disjunction for the argument. Its stepping branch uses `CBVStep.appR`, whose value premise is now available; its value branch satisfies the premise of `CBVStep.beta`. Thus the order of the proof mirrors the order enforced by the reduction constructors.

For a conditional, the induction hypothesis is applied to the guard. A guard step is lifted by `CBVStep.iteC`. Otherwise the Boolean canonical forms lemma, Lemma 3.2, says that the guard is `true` or `false`, so `CBVStep.iteTrue` or `CBVStep.iteFalse` selects the appropriate branch. No branch is evaluated before the guard. This proof therefore establishes progress for the actual evaluation strategy, not merely for arbitrary unrestricted reduction.

The remaining typing cases are direct. Lambda abstractions and Boolean constants are values. The variable case is impossible in an empty context. The complete proof is therefore a structural analysis of a closed typing derivation, with canonical forms used only at the points where a value must be turned into the syntactic form required by a CBV reduction rule.

4.4 Multi-Step Preservation

Evaluation consists of many one-step reductions, so preservation is lifted to the reflexive-transitive closure.

```
theorem preservation_star {G : Context} {M N : Term} {A : Ty}
  (htype : HasType G M A)
  (hsteps : Rewriting.Star CBVStep M N) :
  HasType G N A
```

Theorem 4.4 (Multi-step preservation for CBVStep).

If $\Gamma \vdash M : A$ and $M \rightarrow_{\text{cbv}}^ N$, then $\Gamma \vdash N : A$.*

The proof is an induction on `Rewriting.Star`. In the reflexive case, the original typing proof is the result. In an extension of a path, the induction hypothesis types the intermediate term and one-step `CBVStep.preservation` types the final term.

In particular, the reflexive case covers the possibility that the target is the original term: no reduction need occur before type safety is invoked. In the nontrivial case, the theorem does not reason about an entire evaluation trace at once. It repeats the one-step theorem along the trace supplied by `Rewriting.Star`, preserving the same context and type at every link.

4.5 Call-by-Value Type Safety

Finally, type safety combines multi-step preservation with progress.

```
theorem type_safety {M N : Term} {A : Ty}
  (htype : HasType ([] : Context) M A)
  (hsteps : Rewriting.Star CBVStep M N) :
  Or (IsValue N) (Exists fun P => CBVStep N P)
```

Theorem 4.5 (Type safety for CBVStep).

If $\emptyset \vdash M : A$ and $M \rightarrow_{\text{cbv}}^ N$, then N is a value or there is a P such that $N \rightarrow_{\text{cbv}} P$.*

By Theorem 4.4, the reached term N is still well typed in the empty context. Applying Theorem 4.3 to N gives exactly the conclusion. Thus a closed, well-typed initial term cannot reach a non-value that lacks a call-by-value successor. Together, `CBVStep.to_step`, `CBVStep.preservation`, `CBVStep.progress`, `CBVStep.preservation_star`, and `CBVStep.type_safety` complete the desired call-by-value type-safety layer.

The conclusion is deliberately phrased as no-stuckness rather than as termination. It says that every reachable state is either already an answer or has a legal next call-by-value transition. It does not claim that an evaluation sequence must have finitely many such transitions; that stronger property would require a normalization argument.

5 Discussion

This section discusses the proof structure, explains why progress needs a strategy-specific proof, and records the scope of the result.

5.1 Reusing Unrestricted Metatheory

A central feature of the development is the reuse of the existing unrestricted metatheory. `subject_reduction` already proves preservation for `Step`, but it does not apply directly to a `CBVStep`. The bridge `CBVStep.to_step` turns every call-by-value step into an unrestricted step, after which the existing theorem proves `CBVStep.preservation`.

This suggests a useful pattern for extending formalizations. When a new reduction relation embeds into an existing one, preservation may be inherited instead of reproved constructor by constructor. The pattern is limited to properties that consume an already-given step. Progress asks for the existence of a step, so an embedding into a larger relation does not transfer it in the required direction.

This division of labor keeps the proof both short and honest about what is being inherited. A reduction inclusion gives a way to consume a CBV step with an unrestricted preservation theorem. It cannot manufacture a CBV step from the existence of an unrestricted one, because the latter may choose a redex that the strategy postpones. The proof architecture follows this asymmetry directly.

5.2 Why Progress Needs Its Own Proof

Unrestricted progress says that a closed, well-typed term is a value or has some `Step` successor. Such a successor may be unavailable under the call-by-value scheduling discipline. For an application, the function must be evaluated first, the argument only after the function is a value, and beta reduction only after the argument is also a value. For a conditional, the guard is evaluated before either branch.

Accordingly, `CBVStep.progress` follows the evaluation order directly. It uses `canonical_forms_arr` to turn a value of function type into a lambda abstraction and `canonical_forms_bool` to turn a Boolean value into `true` or `false`. The proof is thus not a cosmetic restatement of unrestricted progress; it captures the operational discipline of `CBVStep`.

The two critical cases also expose why canonical forms are indispensable. When an application function is a value, its arrow type identifies it as a lambda abstraction, rather than merely an arbitrary closed value. When a conditional guard is a value, its Boolean type identifies it as exactly one of the two constants that select a branch. These facts connect static typing to the constructor that provides the next dynamic step.

5.3 Scope and Limitations

The contribution is a type-safety layer for an already defined `CBVStep`; it neither introduces that relation nor changes the syntax or typing of `STLCextBool`. It proves preservation, progress, multi-step preservation, and no-stuckness. It does not prove normalization: type safety does not say that every well-typed term reaches a value in finitely many steps.

The work also does not implement an executable evaluator, prove evaluator soundness or completeness, treat open-term progress, or add an environment semantics. Its language scope is limited to `STLCextBool`; natural numbers, lists, records, exceptions, references, and general recursion are outside it. Finally, this is an operational-metatheory result, not a formalization of denotational or categorical semantics.

The progress theorem is also intentionally stated for terms typed in the empty context, as in the standard type-safety formulation. It does not attempt to make an open term progress without an environment. Similarly, the theorem is about the relation `CBVStep`; an executable interpreter and proofs relating that interpreter to the relation would be a separate development.

5.4 Future Work

One direction is to extend the same development to richer typed languages and to organize the embedding-and-preservation argument as a reusable pattern for other evaluation strategies. Strategy-specific progress proofs will still need to expose their individual evaluation orders.

Another direction is categorical semantics. STLC types can be read as objects, contexts as products, and terms as morphisms. In this view, the Boolean type of `STLCextBool` can be interpreted as $1 + 1$, with `true`, `false`, and conditionals arising from coproduct structure. A broader development could connect the operational results here to a categorical interpretation of `STLCext`, where function types are exponentials, product types are categorical products, sum types are coproducts, and the unit type is terminal.

Such a connection would make it possible to compare operational safety with a structural account of the same typed language. The present result supplies a clean operational starting point for that direction: its type-safety theorems are stated against the repository’s actual syntax, typing relation, and call-by-value reduction rather than against an informal surrogate.

6 Conclusion

We completed the call-by-value type-safety metatheory for the existing `STLCextBool` formalization in the Lean 4 `Metatheory` repository. The existing development already defined `CBVStep`; our contribution is to formalize preservation, progress, multi-step preservation, and type safety for that relation.

First, `CBVStep.to_step` embeds each call-by-value step into the unrestricted relation, allowing `CBVStep.preservation` to reuse the existing `subject_reduction` theorem. Since unrestricted progress does not fix an evaluation order, `CBVStep.progress` is instead proved directly using the existing value and canonical-forms lemmas. We then lift preservation to `Rewriting.Star CBVStep` as `CBVStep.preservation_star` and combine it with progress to obtain `CBVStep.type_safety`.

Consequently, every term reachable by call-by-value evaluation from a closed, well-typed source term is either already a value or can take a further call-by-value step. The formalization therefore rules out stuck states for the existing `STLCextBool` call-by-value semantics. Extending this result to richer languages and connecting it to categorical semantics are natural next steps.

References

- [1] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In André Platzer and Geoff Sutcliffe, editors, *Automated Deduction – CCADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021.
- [2] Benjamin C. Pierce. *Types and Programming Languages*. MIT Press, 2002.
- [3] Benjamin C. Pierce et al. Software foundations. <https://softwarefoundations.cis.upenn.edu/>, 2026. Electronic textbook series; accessed 2026-07-09.
- [4] Arthur Ramos and contributors. Metatheory: A Lean 4 formalization library for programming language metatheory. <https://github.com/Arthur742Ramos/Metatheory>, 2026. GitHub repository; accessed 2026-07-09.

- [5] Arthur Ramos, Anjolina Oliveira, Ruy de Queiroz, and Tiago de Veras. A modular Lean 4 framework for confluence and strong normalization of lambda calculi with products and sums, 2025.
- [6] Philip Wadler, Wen Kokke, and Jeremy G. Siek. Programming language foundations in Agda. <https://plfa.inf.ed.ac.uk/20.07/>, 2020. Version 20.07; accessed 2026-07-09.
- [7] Andrew K. Wright and Matthias Felleisen. A syntactic approach to type soundness. *Information and Computation*, 115(1):38–94, 1994.